

Study_Case1



4143108 - Software Architecture and Design

Nama: Yos Ivan Sirait

NIM: 11423025

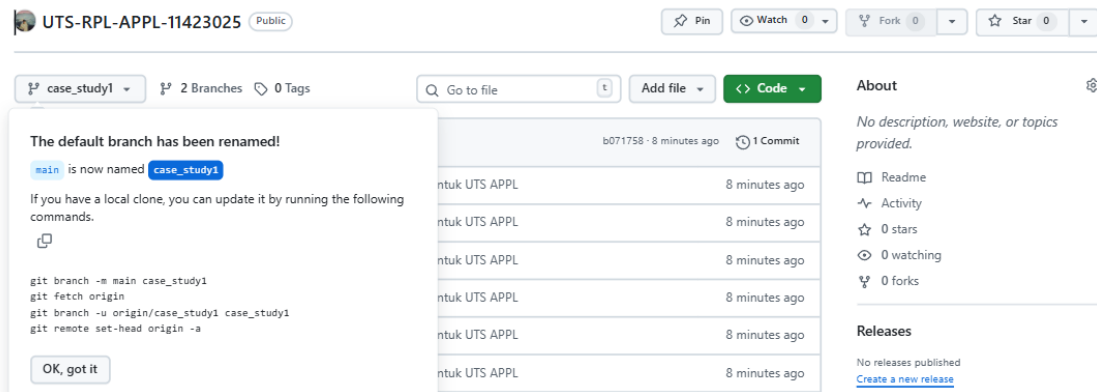
Sarjana Terapan Teknologi Rekayasa

Perangkat Lunak

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

Case Study 1 : "MediTrack - A Digital Healthcare Platform"



1. Arsitektur yang Dipilih

Berdasarkan kebutuhan platform dan ekspektasi pertumbuhan, saya memilih arsitektur Monolitik Modular menggunakan Next.js (App Router) sebagai framework full-stack dan Supabase sebagai Backend-as-a-Service (PostgreSQL + Auth + Storage). Alasan utama: kesederhanaan, kecepatan pengembangan, dan integrasi yang kuat dengan Supabase.

a. Justifikasi dengan Prinsip Arsitektur

1. Kesederhanaan (Simplicity)

- Satu codebase memudahkan pengembangan, debugging, dan deployment.
- Next.js menyediakan API routes dan serverless functions yang terintegrasi, sehingga tim dapat fokus pada fitur tanpa kompleksitas jaringan.

2. Kecepatan Pengembangan (Development Velocity)

- Perubahan lintas modul dapat dilakukan dengan cepat karena semua kode berada dalam satu proyek.
- Supabase mempercepat development dengan menyediakan autentikasi, realtime, dan storage siap pakai.

3. Konsistensi dan Integritas Data (Consistency)

- Menggunakan satu database (Supabase) memastikan transaksi ACID untuk operasi yang melibatkan banyak modul (misal booking appointment yang memerlukan update EHR dan pembayaran).
- Penting untuk data medis yang membutuhkan integritas tinggi.

b. Trade-offs dan Keterbatasan

- Skalabilitas horizontal terbatas – karena satu aplikasi, scaling harus dilakukan pada level instance (load balancing) bukan per modul
- Keterikatan modul – perubahan pada satu modul berpotensi mempengaruhi modul lain. Untuk mengurangi risiko, diterapkan modularisasi dalam struktur folder dan pemisahan tanggung jawab.
- Teknologi terikat – semua harus menggunakan stack yang sama (Next.js, Supabase), tidak bisa memilih teknologi berbeda per modul.

2. System Decomposition & Modeling

Modul	Tanggung Jawab	Entity Utama
IAM (Identity)	Registrasi, Login, dan RBAC (Role-Based Access Control)	User, Role
Appointment	Janji Temu	Appointment, doctor schedule
EHR	Menyimpan Riwayat medis	MedicalRecord
Pharmacy	Pengelolaan resep digital dan stok obat	Medication Stock
Payment	Payment Gateway	Payment

3. System Decomposition & Modeling

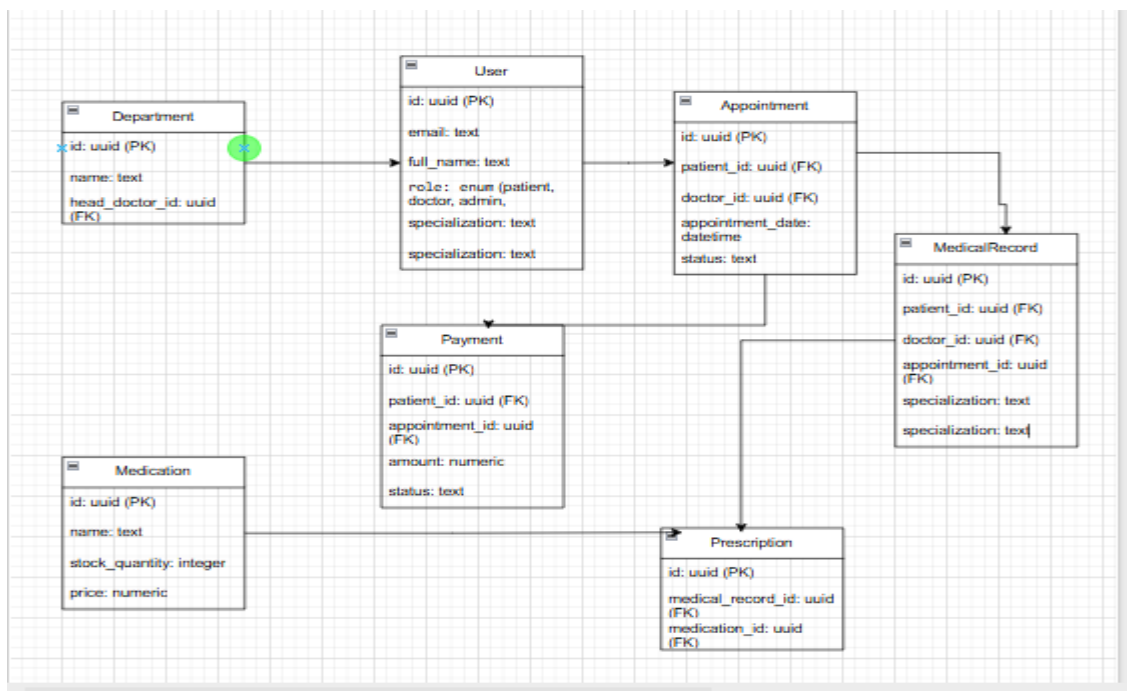
1. Break down the MediTrack system into core modules or services based on your chosen architecture.

```
meditrack/
├── app/
│   ├── api/
│   │   ├── auth/           # Authentication endpoints
│   │   ├── users/          # User management
│   │   ├── appointments/   # Appointment management
│   │   ├── medical-records/ # EHR system
│   │   ├── pharmacy/
│   │   │   ├── medications/ # Medication management
│   │   │   └── prescriptions/ # Prescription management
│   │   ├── payments/       # Payment processing
│   │   └── analytics/      # Analytics data
│   ├── auth/
│   │   ├── login/          # Login page
│   │   └── register/       # Registration page
│   ├── dashboard/
│   │   ├── patient/        # Patient dashboard
│   │   ├── doctor/         # Doctor dashboard
│   │   └── admin/          # Admin dashboard
│   ├── globals.css         # Global styles & theme
│   ├── layout.tsx          # Root layout
│   └── page.tsx             # Home page
├── components/
│   └── ui/                  # shadcn/ui components
├── lib/
│   ├── db.ts               # Database utilities
│   ├── auth.ts             # Authentication utilities
│   ├── api.ts              # API helpers
│   ├── validators.ts       # Zod validators
│   └── utils.ts            # General utilities
├── types/
│   └── index.ts             # TypeScript type definitions
├── scripts/
│   └── 01-init-schema.sql   # Database schema
├── package.json
├── tsconfig.json
└── tailwind.config.ts
```

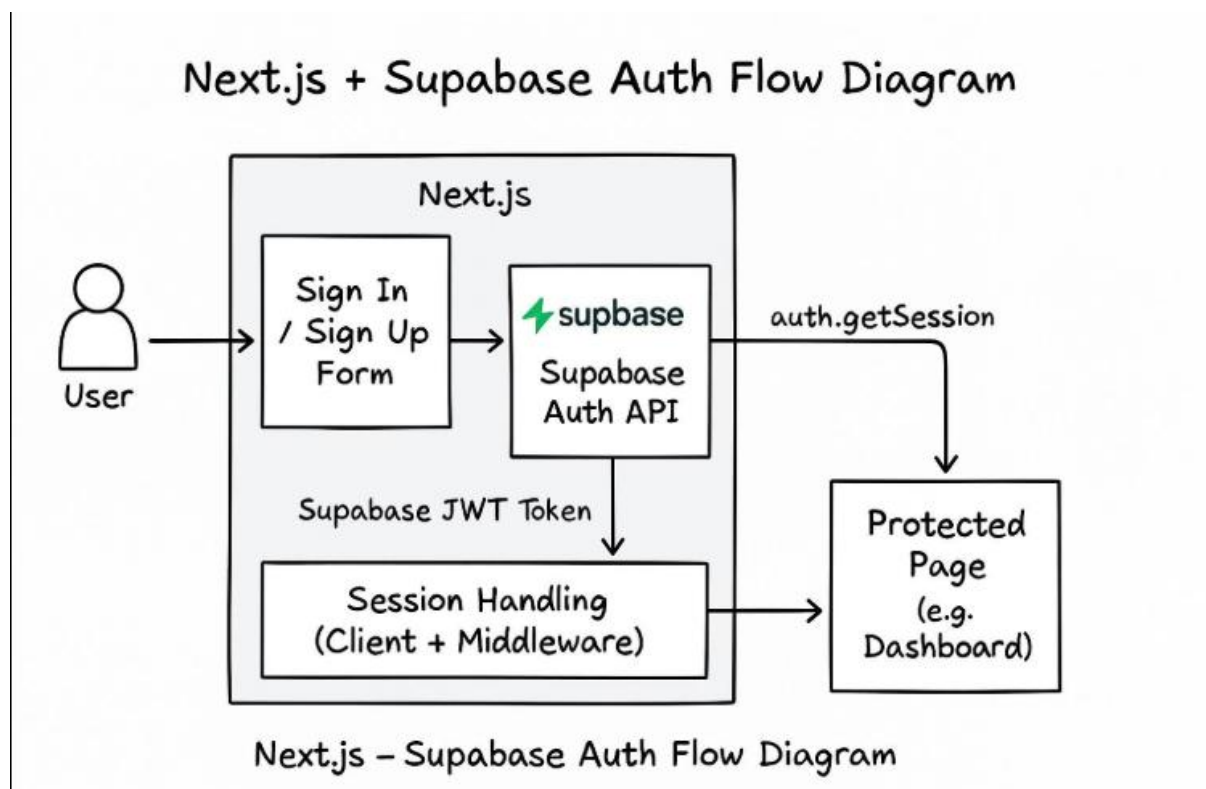
2. Relationships Diagram



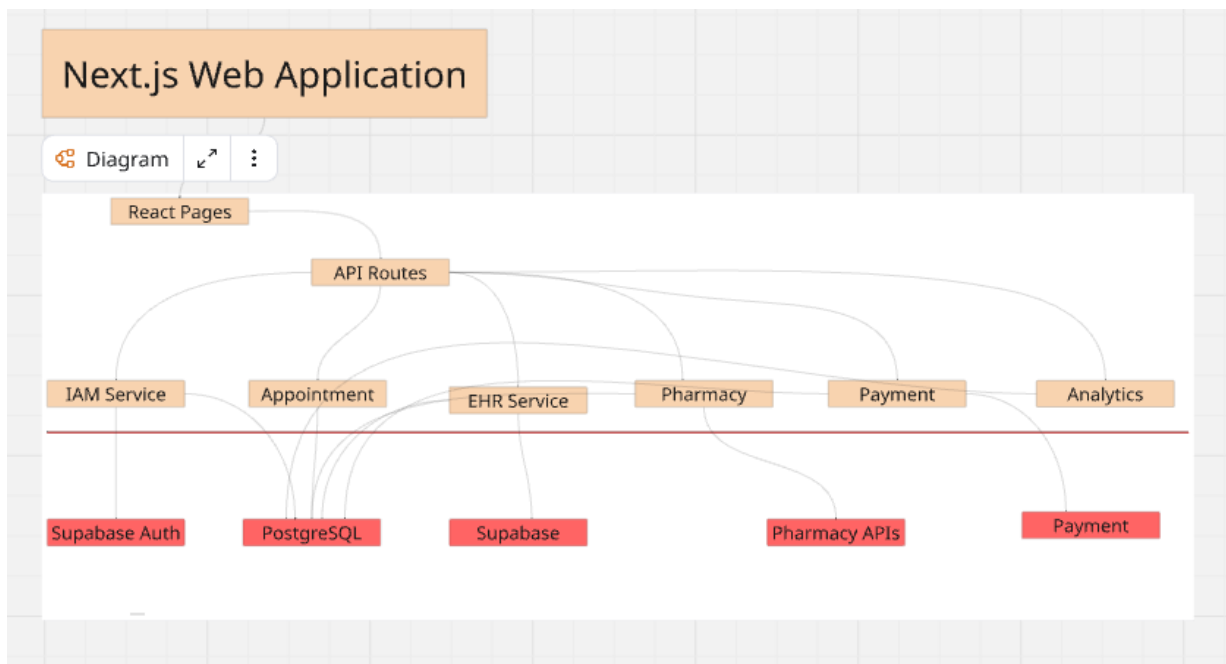
3. Use UML class diagrams or component diagrams to illustrate the design.



4. Architecture Visualization



Alur kerja sistem MediTrack dimulai ketika pengguna melakukan interaksi melalui Web Client berbasis Next.js, di mana permintaan (request) akan melewati lapisan Middleware untuk proses autentikasi dan pengecekan peran (RBAC) sebelum diteruskan ke API Routes yang terorganisir secara modular. Setiap rute API kemudian memanggil layanan bisnis yang spesifik—seperti Appointment Service atau EHR Service—yang bertugas mengolah logika data dengan berinteraksi langsung ke Supabase untuk autentikasi identitas dan penyimpanan data pada database PostgreSQL



Batasan Arsitektur dan Interaksi Sistem

1. **Batasan Modular (Modular Boundaries)** Dalam aplikasi Next.js ini, setiap modul (IAM, Appointment, EHR, dll.) dipisahkan secara logis ke dalam direktorinya masing-masing di bawah `lib/services/` atau `app/api/`. Hal ini memastikan bahwa logika bisnis terenkapsulasi di dalam modulnya sendiri. Komunikasi antar-modul ditangani melalui panggilan layanan internal atau antarmuka bersama (*shared interfaces*) yang terdefinisi dengan baik. Pendekatan ini mencegah akses database lintas-modul secara langsung sedapat mungkin untuk menjaga integritas modular.

2. **Batasan Lapisan Layanan (Service Layer Boundaries)** Lapisan layanan (*Service Layer*) bertindak sebagai pusat logika bisnis inti. Lapisan ini mengabstraksi akses data yang mendasari (*Supabase/PostgreSQL*) dan integrasi eksternal, sehingga menyediakan API yang bersih untuk dikonsumsi oleh frontend dan rute API (*API routes*). Pemisahan ini memastikan bahwa perubahan pada skema database atau penyedia layanan eksternal tidak berdampak langsung pada antarmuka pengguna (*User Interface*).

3. Aliran Data (Data Flow)

Berikut adalah urutan aliran data dalam sistem MediTrack:

1. **Aksi Pengguna:** Pengguna berinteraksi dengan Web Client (misalnya, melakukan pemesanan janji temu).

2. Permintaan (Request): Frontend mengirimkan permintaan ke rute API (API Route) yang sesuai.
3. Logika Bisnis: Rute API memanggil Layanan (Service) terkait (misalnya, Appointment Service).
4. Penyimpanan/Pengambilan Data: Layanan berinteraksi dengan Supabase Auth untuk identitas atau Database PostgreSQL untuk operasi data.
5. Integrasi Eksternal: Jika diperlukan, Layanan berkomunikasi dengan Sistem Eksternal (misalnya, memproses pembayaran melalui gateway).
6. Respons: Hasil dikirimkan kembali melalui Rute API ke Web Client, yang kemudian memperbarui tampilan antarmuka (UI).

5. Performance Bottlenecks & Mitigation Strategies

Sebagai sistem yang dipersiapkan untuk pertumbuhan cepat, beberapa potensi hambatan kinerja telah diidentifikasi beserta strategi mitigasinya:

- Database Connection Overload: Karena menggunakan arsitektur monolitik dengan banyak modul, beban pada satu database PostgreSQL bisa meningkat drastis.
 - ⇒ Mitigasi: Mengaktifkan Connection Pooling (menggunakan fitur Supabase/Prisma Accelerate) untuk x mengelola koneksi database secara efisien.

Study_Case2

1. Arsitektur Microservices Ecosystem

Berdasarkan kebutuhan untuk scalability (skalabilitas) dan high availability (ketersediaan tinggi), MediTrack bertransformasi dari Java Monolith ke Spring Cloud Microservices.

- Justifikasi dengan Prinsip Arsitektur:
 - a. Single Responsibility Principle (SRP): Setiap layanan (Auth, Appointment, EHR, dll) hanya bertanggung jawab atas satu domain bisnis. Jika modul Payment mengalami lonjakan transaksi, kita hanya perlu menambah kapasitas (scaling) pada Payment Service tanpa menyentuh modul Pharmacy.
 - b. Database-per-Service (Data Isolation): Setiap microservice memiliki database sendiri. EHR menggunakan MongoDB karena data rekam medis bersifat tidak terstruktur (dokumen), sedangkan Auth menggunakan PostgreSQL untuk integritas data pengguna yang kaku. Ini mencegah Single Point of Failure pada database.
 - c. Loose Coupling & Event-Driven: Antar layanan tidak lagi memanggil fungsi secara langsung (Direct Method Call), melainkan melalui REST API (Synchronous) untuk kebutuhan cepat, dan Message Queue/Kafka (Asynchronous) untuk proses latar belakang.
Contoh: Saat janji temu dibuat di Appointment Service, sebuah pesan dikirim ke Kafka. EHR Service dan Analytics Service akan mengambil pesan tersebut untuk memperbarui data secara otomatis tanpa membuat user menunggu.

2. Use Case Analysis

Bertransformasi dari sistem monolitik yang kaku menjadi ekosistem microservices.

Berikut adalah fungsionalitas utama yang dipertahankan:

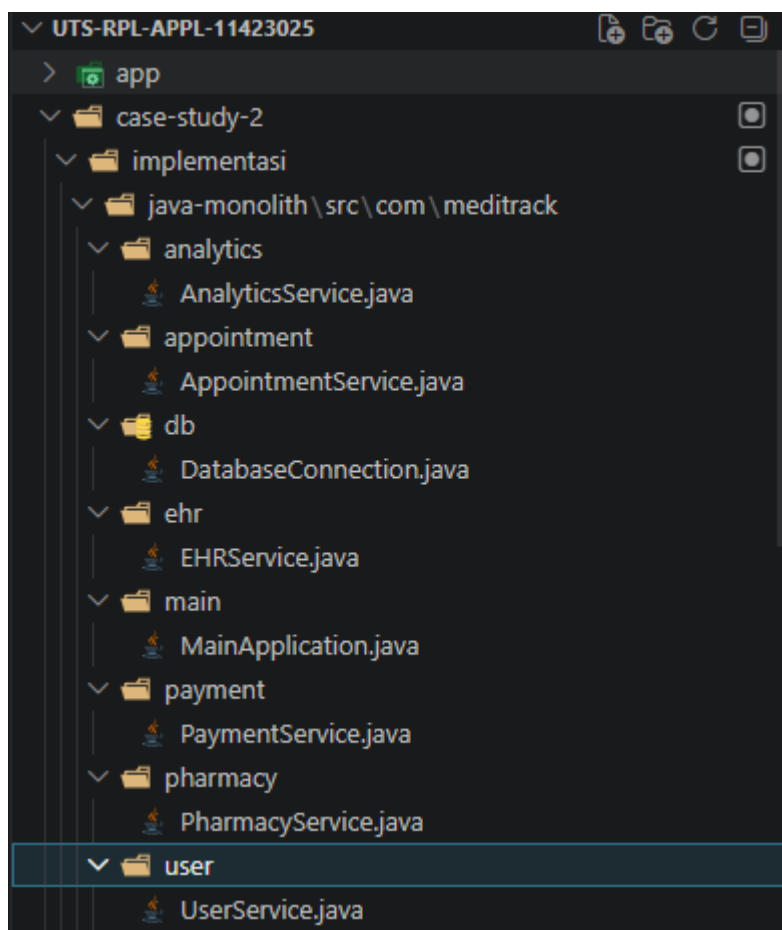
- Auth & User Service: Mengelola identitas (Pasien, Dokter, Admin) dan hak akses (RBAC).
- Appointment Service: Menangani penjadwalan janji temu antara pasien dan dokter.

- EHR (Electronic Health Records): Mengelola data medis sensitif dan riwayat kesehatan.
- Pharmacy Service: Manajemen stok obat dan pemrosesan resep digital.
- Payment Service: Gerbang pembayaran untuk layanan medis dan obat-obatan.
- Analytics Service: Pengolahan data besar untuk tren kesehatan dan performa operasional.

3. Architecture Diagram

A. Monolithic Architecture

Seluruh modul berada dalam satu proses (Single Runtime) dan berbagi database yang sama.



B. Microservice Archetecture

Modul terpisah menjadi layanan independent yang terisolasi

Struktur Microservices

```
case-study-2/implementasi/microservices/  
├─ auth-service/           # Service untuk autentikasi & manajemen user  
├─ appointment-service/    # Service untuk penjadwalan janji temu  
├─ ehr-service/            # Service untuk Electronic Health Records  
├─ pharmacy-service/       # Service untuk manajemen farmasi  
├─ analytics-service/      # Service untuk analisis data  
└─ payment-service/        # Service untuk pemrosesan pembayaran
```

```
microservices  
├── analytics-service\src\main\java\com\meditrack\analytics  
│   ├── AnalyticsApplication.java  
│   └── AnalyticsController.java  
├── appointment-service\src\main\java\com\meditrack\appointment  
│   ├── AppointmentApplication.java  
│   └── AppointmentController.java  
├── auth-service\src\main\java\com\meditrack\auth  
│   ├── AuthApplication.java  
│   └── AuthController.java  
├── ehr-service\src\main\java\com\meditrack\ehr  
│   ├── EHRApplication.java  
│   └── EHRController.java  
├── payment-service\src\main\java\com\meditrack\payment  
│   ├── PaymentApplication.java  
│   └── PaymentController.java  
└── pharmacy-service\src\main\java\com\meditrack\pharmacy  
    ├── PharmacyApplication.java  
    └── PharmacyController.java
```

4. Migration Strategi

Menggunakan strategi migrasi bertahap untuk meminimalkan downtime:

- Phase 1: Proxying. Memasang API Gateway di depan Monolith.
- Phase 2: Extraction. Memindahkan modul dengan risiko terkecil (misal: Analytics) ke service baru. Phase
- 3: Service Discovery. Menggunakan Eureka agar service baru bisa saling menemukan.
- Phase 4: Decommission. Mematikan modul di Monolith satu per satu setelah service baru stabil.

5. Reasoning & Trade-offs

Aspek	Alasan Transformasi	Trade-off (Risiko)
Maintainability	Tim kecil bisa focus pada satu service tanpa merusak kode	Kompleksitas tracing (sulit melacak error antar error)
Resilience	Jika salah satu modul down, maka modul lainnya bisa tetap berjalan	Membutuhkan mekanisme Circuit Breaker agar system tidak cascade failure
Data Consistency	Data terisolasi, meningkatkan keamanan dan integrasi permodul	Database terpisah , sinkronisasi antar modu tidak mudah/instan
Deployment	Update fitur Analytics tidak perlu me-restart seluruh system Meditrack	Pipeline CI/CD lebih kompleks